

Factullama

Guía de Preparación para Entrevista Técnica

Contexto técnico del proyecto + banco de preguntas y respuestas:
Java & Spring Boot · Spring Security · LRBA (BBVA) · APX (BBVA)

Preparado para: Divtyre
monkeycoffedevs@gmail.com
29 de junio de 2026

Contenido

- 1. Resumen técnico del proyecto Factullama
- 2. Java & Spring Boot — Preguntas y respuestas
- 3. Spring Security — Preguntas y respuestas
- 4. LRBA (BBVA) — Preguntas y respuestas
- 5. APX / Arquitectura Plataforma eXtendida (BBVA) — Preguntas y respuestas
- 6. Cómo hablar de LRBA y APX sin experiencia directa

1. Resumen técnico del proyecto Factullama

1.1 Qué es el proyecto (elevator pitch)

Factullama es un SaaS multi-tenant pensado para pequeñas empresas peruanas (restaurantes y comercios) que unifica cuatro frentes en un solo sistema: facturación electrónica alineada a SUNAT (XML UBL 2.1 firmado digitalmente), un punto de venta para restaurantes (mesas, pisos y caja), un flujo de carrito/checkout para venta de productos, y suscripciones de pago vía MercadoPago. Incluye además un CRM básico (clientes y cotizaciones).

Frase de presentación lista para usar: «Es una plataforma SaaS para pequeñas empresas peruanas que combina facturación electrónica, punto de venta y gestión comercial en un solo sistema, con aislamiento de datos por empresa (multi-tenant) y autenticación stateless con JWT.»

1.2 Stack tecnológico

Capa	Tecnologías
Backend	Java 21, Spring Boot 3.2.5, Spring Web, Spring Data JPA + Hibernate, H2 (dev) / MariaDB (prod), Lombok, springdoc-openapi 2.3.0 (Swagger UI), Maven.
Seguridad	Spring Security, JWT vía jjwt 0.12.3 (firma HMAC-SHA256), BCryptPasswordEncoder, filtro OncePerRequestFilter, sesiones 100% stateless.
Integraciones externas	MercadoPago SDK 3.2.1 (Preference API) para pagos/suscripciones; ApiPeru.dev (RestTemplate + Bearer token) para validar DNI/RUC; SUNAT — XML UBL 2.1 construido a mano, firmado con Apache Santuario (xmlsec 3.0.2) + BouncyCastle 1.77 sobre un certificado PKCS12 (.pfx).
Frontend	Angular 21 (standalone components, rutas con lazy loading), PrimeNG, interceptor HTTP para adjuntar el JWT, route guards para proteger el shell /app.
Infraestructura	Docker Compose (backend, frontend, nginx), Nginx como reverse proxy, despliegue vía Dokploy sobre una red Docker externa (dokploy-network), MariaDB en producción.

1.3 Arquitectura y patrones notables

Capas clásicas Spring. Controller → Service → Repository (JPA), con DTOs para request/response y un único **@RestControllerAdvice** (GlobalExceptionHandler) que centraliza el manejo de errores: NotFoundException → 404, BadRequestException → 400, MethodArgumentNotValidException → 400 con el primer mensaje de validación, y un catch-all → 500. Todas las respuestas de error siguen el mismo contrato (ApiError: timestamp, status, error, message, path).

Multi-tenancy «ligero». Cada User tiene una Empresa asociada 1 a 1 (OneToOne mappedBy "owner"). Para aislar datos por empresa, el proyecto repite un método privado **requireEmpresa(user)** en cada servicio (Factura, Mesa, Producto, Cliente, Cotización). Este es el cuerpo real, idéntico en los cinco servicios:

```
private Empresa requireEmpresa(User user) {  
    if (user.getEmpresa() == null)  
        throw new BadRequestException("El usuario no tiene empresa asociada");  
    return user.getEmpresa();  
}
```

Es un patrón honesto para discutir en la entrevista: funciona y es fácil de leer, pero está duplicado en cinco clases, lo que viola DRY y es fácil de olvidar al agregar un servicio nuevo. Una mejora real sería moverlo a una clase base, a un componente compartido inyectado, o resolver la empresa actual directamente desde el SecurityContext con una anotación personalizada (@CurrentEmpresa) y un resolver de argumentos.

Inconsistencia real para mencionar con honestidad

OrdenService.findForUser() filtra por user.getId() en vez de por empresa, mientras que el resto de servicios (Factura, Mesa, Producto, Cliente) filtran por empresa. Son dos estrategias de aislamiento de datos distintas conviviendo en el mismo código base — un excelente ejemplo real, no inventado, para responder a «contame un bug o inconsistencia que encontraste en tu propio código» mostrando capacidad de autocritica técnica.

Seguridad. SecurityConfig define una cadena de filtros completamente stateless: CSRF deshabilitado (justificado porque no se usan cookies de sesión, solo Bearer tokens), rutas públicas explícitas (/api/auth/**, /api/test/**, /api/v1/consulta/**, /api/v1/pagos/**) y un JwtAuthenticationFilter insertado antes de UsernamePasswordAuthenticationFilter. El filtro extrae el JWT del header Authorization, obtiene el username, carga el UserDetails desde UserRepository, valida el token y puebla el SecurityContextHolder. JwtService firma con HMAC-SHA256 y agrega el primer "granted authority" del usuario como claim "role" dentro del token. La clase User implementa UserDetails directamente (en vez de tener un adaptador separado), lo cual simplifica el código pero acopla el modelo de dominio a la interfaz de Spring Security.

Facturación electrónica (SUNAT). SunatService construye manualmente el XML UBL 2.1 para Factura/Boleta (con StringBuilder), lo firma con FirmadorService usando Apache Santuario (algoritmo RSA-SHA256, transform de firma enveloped) y comprime el resultado — pero el envío final al webservice real de SUNAT está simulado (el método devuelve un mensaje fijo de "pendiente de configuración"). FirmadorService carga el certificado .pfx en un @PostConstruct; si la carga falla, no rompe el arranque de la aplicación, solo deshabilita la firma y registra una advertencia — un buen ejemplo real de *graceful degradation* para mencionar en entrevista.

Pagos y checkout. La integración con MercadoPago genera un init_point de checkout vía su SDK Java; el dominio Pago tiene estados (PENDIENTE/CONFIRMADO) y se referencia desde Orden. OrdenService.checkout() valida que el carrito no esté vacío, calcula IGV (18% fijo) y total con BigDecimal y RoundingMode.HALF_UP (nunca double, para evitar errores de redondeo en dinero), decrementa stock con validación de insuficiencia, crea el Pago en PENDIENTE y limpia el carrito. Dato curioso: Empresa no tiene un campo de plan/suscripción explícito, aunque la lógica de pagos sugiere un modelo de suscripción — posible deuda técnica o feature a medio terminar.

Sin pruebas automatizadas ni CI/CD. No se encontraron clases JUnit (solo un TestController, que es un endpoint REST, no un test) ni carpetas .github/workflows. Es un punto honesto para la pregunta «¿qué mejorarías?»: agregar tests unitarios de servicios con Mockito, tests de integración con @SpringBootTest + Testcontainers, y un pipeline de CI que al menos compile y testee en cada push.

1.4 Hallazgos reales para discutir (la honestidad técnica es una señal positiva)

Secretos hardcodeados en el repositorio

application-prod.properties contiene en texto plano la contraseña de la base de datos MariaDB y el secreto usado para firmar los JWT, committeados al control de versiones. Riesgo real de seguridad; la corrección esperable es moverlos a variables de entorno o a un gestor de secretos (Docker secrets, Vault, AWS/GCP Secret Manager) y nunca versionarlos.

CORS con dos dominios distintos

SecurityConfig.java permite el origen `https://factullama.site`, pero `application-prod.properties` define `https://mantaro.com` como origen permitido. Dos configuraciones de CORS divergentes — posible bug latente o resabio de un rebranding del producto.

1.5 Cómo contar el proyecto en 60 segundos

«Desarrollé Factullama, un SaaS para pequeñas empresas peruanas. El backend es Java 21 con Spring Boot 3.2.5: expone una API REST securizada con Spring Security y JWT stateless, persiste con Spring Data JPA sobre MariaDB, y aísla los datos de cada empresa a nivel de aplicación. El módulo más interesante es la facturación electrónica: genero el XML UBL 2.1 de la factura, lo firmo digitalmente con un certificado PKCS12 usando Apache Santuario, y dejé preparado el flujo de envío a SUNAT. También integré MercadoPago para las suscripciones y construí un punto de venta para restaurantes con gestión de mesas y caja. El frontend es Angular con componentes standalone y carga perezosa por ruta. Aprendí mucho sobre multi-tenancy, manejo seguro de criptografía en Java, y también identifiqué deuda técnica real como falta de tests, que sería lo primero que atacaría si siguiera el proyecto.»

2. Java & Spring Boot — Preguntas y respuestas

Banco de preguntas frecuentes en entrevistas de backend Java/Spring Boot, con respuesta completa para cada una. Donde aplica, se conecta con código real de Factullama.

P1. ¿Qué diferencia hay entre Spring Framework y Spring Boot?

R: Spring Framework es el contenedor de inversión de control (IoC) y el conjunto de módulos (MVC, Data, Security, AOP, etc.) que resuelven problemas concretos de una aplicación empresarial. Spring Boot se construye encima: añade autoconfiguración basada en las dependencias presentes en el classpath, un servidor embebido (Tomcat por defecto), starters que agrupan dependencias coherentes (spring-boot-starter-web, -security, -data-jpa) y opinión por convención para reducir XML y configuración manual. En Factullama, por ejemplo, solo con incluir spring-boot-starter-data-jpa y definir spring.datasource.url ya tengo un EntityManager y un DataSource configurados sin escribir una sola clase de configuración.

P2. ¿Qué es la inversión de control (IoC) y la inyección de dependencias (DI)?

R: IoC es el principio de que el framework, y no la clase, decide cuándo y cómo se crean e interconectan los objetos. DI es la técnica concreta con la que Spring aplica IoC: en vez de que una clase haga "new" de sus dependencias, las recibe ya construidas, normalmente por constructor. En Factullama todos los servicios usan inyección por constructor: por ejemplo AuthService recibe AuthenticationManager, JwtService, UserRepository y PasswordEncoder en su constructor, sin anotación @Autowired explícita porque desde Spring 4.3 es opcional con un único constructor.

P3. ¿Qué tipos de inyección de dependencias existen en Spring y cuál preferís?

R: Por constructor, por setter y por campo (@Autowired directo sobre el atributo). Prefiero inyección por constructor: hace que las dependencias sean explícitas e inmutables (se pueden declarar final), permite detectar dependencias circulares en tiempo de arranque en vez de en producción, y facilita los tests unitarios porque puedo instanciar la clase con mocks sin necesidad de un contenedor Spring. Es exactamente el patrón que usa todo el backend de Factullama.

P4. ¿Qué hace la anotación @SpringBootApplication?

R: Es una anotación compuesta que agrupa tres: @Configuration (la clase puede declarar @Bean), @EnableAutoConfiguration (activa la autoconfiguración basada en las dependencias del classpath) y @ComponentScan (escanea el paquete actual y subpaquetes en busca de @Component, @Service, @Repository, @Controller). Por eso basta con tener una clase con esta anotación en el paquete raíz (com.facturacion en este proyecto) para que todo el resto de clases anotadas se registre automáticamente.

P5. ¿Qué diferencia hay entre @Component, @Service, @Repository y @Controller/@RestController?

R: Las cuatro son especializaciones de @Component y, en términos de funcionamiento del contenedor, se comportan igual: registran un bean. La diferencia es semántica y de herramientas: @Service marca lógica de negocio (FacturaService, OrdenService), @Repository marca acceso a datos y activa traducción de excepciones de persistencia a excepciones de Spring (DataAccessException), y @Controller/@RestController marcan la capa web — @RestController añade @ResponseBody implícito, serializando el retorno directo a JSON, que es lo que usa Factullama en todos sus controllers.

P6. ¿Qué es un Spring Bean y cuáles son sus scopes principales?

R: Un bean es cualquier objeto cuyo ciclo de vida (creación, configuración, destrucción) gestiona el contenedor de Spring en vez de gestionarlo el código de la aplicación. El scope por defecto es singleton: una única instancia compartida por todo el contexto, que es lo que usan FacturaService, JwtService, etc. Otros scopes son prototype (una instancia nueva cada vez que se inyecta), request y session (uno por petición HTTP o por sesión, típico en apps web), y application. En una API stateless como Factullama casi todo es singleton porque los servicios no guardan estado mutable por usuario.

P7. ¿Cómo gestiona Spring Boot la configuración por entornos (profiles)?

R: Con archivos application-{profile}.properties o .yaml y la propiedad spring.profiles.active. Factullama tiene application.yml para desarrollo (H2 en memoria, JWT de 1 hora, CORS habilitado) y application-prod.properties para producción (MariaDB real, logging más silencioso). Al desplegar se activa el perfil prod y Spring sobrescribe las propiedades compartidas con las específicas del perfil. Las propiedades sensibles (password de BD, secreto JWT) deberían inyectarse como variables de entorno con la sintaxis \${VARIABLE} en vez de quedar escritas en el archivo, que es justamente el problema que encontré en el application-prod.properties de este proyecto.

P8. ¿Qué es Spring Data JPA y cómo simplifica el acceso a datos?

R: Es el módulo que evita escribir implementaciones de DAO a mano: defino una interfaz que extiende JpaRepository y Spring genera en tiempo de ejecución una implementación con operaciones CRUD, paginación y ordenamiento. También permite derivar queries del nombre del método (findByEmpresaOrderByCreatedAtDesc, que usa FacturaRepository) o escribir JPQL/SQL nativo con @Query cuando la derivación no alcanza. Reduce drásticamente el código repetitivo de acceso a datos.

P9. ¿Qué diferencia hay entre FetchType.LAZY y EAGER, y qué es el problema N+1?

R: EAGER carga la asociación inmediatamente junto con la entidad principal; LAZY la carga solo cuando se accede a ella por primera vez, mediante un proxy. LAZY es la opción por defecto recomendada para colecciones y relaciones OneToMany/ManyToOne porque evita traer datos que no se van a usar. El problema N+1 ocurre cuando, por ejemplo, listo N facturas y por cada una Hibernate dispara una query adicional para cargar su relación lazy (sus items), terminando con N+1 queries en vez de una sola con JOIN. Se soluciona con JOIN FETCH en la query, con @EntityGraph, o con proyecciones DTO que traen todo en una sola consulta.

P10. ¿Qué hace @Transactional y qué pasa si ocurre una excepción dentro de un método anotado así?

R: Envuelve el método en una transacción de base de datos: si todo sale bien, hace commit al finalizar; si se lanza una excepción no controlada (RuntimeException o una marcada explícitamente en rollbackFor), Spring revierte (rollback) todos los cambios hechos en esa transacción, manteniendo la consistencia de los datos. FacturaService.crear() está anotado @Transactional: si falla a mitad de construir los items de la factura, ninguna fila parcial queda guardada. @Transactional(readOnly = true), como en listar(), además permite optimizaciones del proveedor JPA para consultas de solo lectura.

P11. ¿Cómo se maneja el manejo centralizado de errores en una API REST con Spring?

R: Con una clase anotada @RestControllerAdvice junto con métodos @ExceptionHandler para cada tipo de excepción. En Factullama, GlobalExceptionHandler intercepta NotFoundException (404), BadRequestException (400), MethodArgumentNotValidException —de @Valid en los DTOs— (400 con el primer mensaje de validación) y cualquier Exception genérica como fallback (500). Esto evita repetir try/catch en cada controller y asegura que toda la API devuelva errores con la misma forma (un objeto ApiError con timestamp, status, error, message y path).

P12. ¿Qué es el patrón DTO y por qué no conviene exponer las entidades JPA directamente?

R: Un DTO (Data Transfer Object) es un objeto plano usado solo para mover datos entre capas, típicamente entre el controller y el cliente HTTP. Exponer entidades JPA directamente acopla el contrato de la API a la estructura de la base de datos, puede filtrar datos sensibles (passwords, relaciones internas), y genera errores de serialización con relaciones lazy fuera de una sesión activa (LazyInitializationException). FacturaResponse, LoginResponse y UserInfo en Factullama son DTOs que controlan exactamente qué campos salen al cliente.

P13. ¿Diferencia entre List, Set y Map, y cuándo usarías cada uno?

R: List mantiene orden de inserción y permite duplicados (ArrayList para acceso aleatorio rápido, LinkedList para inserciones/eliminaciones frecuentes en los extremos). Set no permite duplicados (HashSet para búsqueda O(1) sin orden, LinkedHashSet con orden de inserción, TreeSet ordenado). Map asocia claves únicas a valores (HashMap para la mayoría de los casos, LinkedHashMap si el orden importa, TreeMap si necesito las claves ordenadas). En Factullama, por ejemplo, los items de un carrito o una factura se modelan como List porque el orden de los items y la posibilidad de repetir un mismo producto en distintas líneas son relevantes.

P14. ¿Qué son los Streams en Java? Dá un ejemplo.

R: Es una API funcional para procesar colecciones de forma declarativa: en vez de iterar con for, se encadenan operaciones intermedias (map, filter, sorted) y una operación terminal (collect, forEach, toList). Son inmutables respecto a la fuente y favorecen código más legible. En FacturaService se usa exactamente este patrón: `facturaRepository.findByEmpresaOrderByCreatedAtDesc(empresa).stream().map(FacturaResponse::new).toList()` transforma una lista de entidades Factura en una lista de DTOs FacturaResponse en una sola línea legible.

P15. ¿Por qué usar BigDecimal en vez de double para cálculos de dinero?

R: double y float usan representación binaria de punto flotante, que no puede representar exactamente la mayoría de los decimales ($0.1 + 0.2$ no da exactamente 0.3), lo que en cálculos financieros repetidos acumula errores de redondeo. BigDecimal representa el número de forma exacta como un entero más una escala, y permite elegir explícitamente el modo de redondeo. Factullama calcula el IGV y los totales de factura con BigDecimal y RoundingMode.HALF_UP, que es el comportamiento correcto y esperado para un sistema de facturación real.

P16. ¿Qué diferencia hay entre excepciones checked y unchecked en Java?

R: Las checked (que extienden Exception pero no RuntimeException) obligan al compilador a exigir que se declaren con throws o se capturen con try/catch — representan condiciones que el llamador razonablemente puede manejar, como IOException. Las unchecked (extienden RuntimeException) no obligan a declararlas y suelen representar errores de programación o de negocio que se resuelven con manejo centralizado. Factullama usa excepciones unchecked propias (NotFoundException, BadRequestException) precisamente para no ensuciar las firmas de los métodos de servicio y delegar el manejo al GlobalExceptionHandler.

P17. ¿Cómo testearías un servicio Spring? ¿Qué rol juega Mockito?

R: Para un test unitario, instancio el servicio directamente con JUnit (sin levantar el contexto de Spring) y le paso mocks de sus dependencias creados con Mockito (@Mock, @InjectMocks), definiendo comportamientos con when(...).thenReturn(...) y verificando interacciones con verify(...). Para tests de integración, uso @SpringBootTest (levanta el contexto real) combinado con una base de datos de prueba, idealmente con Testcontainers para no depender de H2 en memoria si quiero fidelidad con MariaDB. Reconozco abiertamente que Factullama no tiene esta capa hoy — es la primera mejora que propondría.

P18. ¿Qué es OpenAPI/Swagger y por qué lo usarías en un proyecto backend?

R: OpenAPI es una especificación estándar para describir una API REST (endpoints, parámetros, esquemas de request/response, códigos de estado) de forma legible por máquinas y humanos. springdoc-openapi genera esa especificación automáticamente a partir del código y expone una UI interactiva (Swagger UI) donde se puede probar cada endpoint sin un cliente externo. Factullama incluye springdoc-openapi-starter-webmvc-ui, lo que facilita que el frontend Angular y cualquier consumidor externo entiendan el contrato de la API sin depender de documentación manual desactualizada.

3. Spring Security — Preguntas y respuestas

Esta sección está directamente respaldada por la implementación real de Factullama (SecurityConfig, JwtAuthenticationFilter, JwtService, User implementando UserDetails), así que podés responder con ejemplos concretos de tu propio código.

P1. ¿Cómo funciona la cadena de filtros (FilterChain) de Spring Security?

R: Cada request HTTP pasa por una cadena ordenada de filtros antes de llegar al controller. Cada filtro puede inspeccionar, modificar o cortar la cadena (por ejemplo, devolviendo 401 sin llegar al controller). En Factullama, SecurityConfig registra JwtAuthenticationFilter (addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class)): este filtro extiende OncePerRequestFilter (se ejecuta una sola vez por request), extrae el JWT del header Authorization, valida el usuario y, si todo es correcto, puebla el SecurityContextHolder antes de que la petición siga su camino hacia el controller correspondiente.

P2. ¿Qué diferencia hay entre autenticación y autorización?

R: Autenticación responde «¿quién sos?»: verificar que las credenciales (en este caso, email + password contra el hash BCrypt almacenado) correspondan a un usuario real. Autorización responde «¿qué podés hacer?»: una vez autenticado, decidir si ese usuario tiene permiso para una acción concreta, normalmente en base a sus authorities/roles (ROLE_ADMIN, ROLE_USER en este proyecto). AuthService.authenticate() resuelve la autenticación delegando en AuthenticationManager; la autorización se resolvería con @PreAuthorize o reglas en SecurityConfig sobre rutas específicas.

P3. ¿Por qué usar JWT en vez de sesiones tradicionales? Ventajas y desventajas.

R: JWT permite que el backend sea completamente stateless: no hay que guardar sesión en memoria ni en una tabla compartida, lo que facilita escalar horizontalmente sin sticky sessions ni un store de sesión centralizado (Redis, etc.). El servidor solo necesita la clave de firma para validar que un token no fue alterado. La desventaja principal es que, al no haber estado en el servidor, revocar un token antes de su expiración natural es difícil (no hay un "logout" real del lado del servidor salvo que se implemente una blacklist). En Factullama los tokens expiran en 1 hora en desarrollo, lo que limita la ventana de riesgo.

P4. ¿Cómo funciona un JWT internamente?

R: Un JWT tiene tres partes separadas por puntos, cada una en Base64Url: header (algoritmo de firma, ej. HS256, y tipo), payload (claims: subject, fecha de emisión, expiración, y claims propios como "role" en este proyecto) y signature (el hash HMAC del header+payload usando una clave secreta, que permite detectar si el token fue modificado). JwtService.generateToken() construye exactamente esto con la librería jjwt: setClaims, setSubject(email), setIssuedAt, setExpiration y signWith(signingKey, HS256).

P5. ¿Qué son UserDetails y UserDetailsService?

R: UserDetails es la interfaz que Spring Security usa para representar a un usuario autenticable: expone username, password y authorities, además de flags de cuenta (habilitada, no expirada, no bloqueada). UserDetailsService es el contrato con un único método, loadUserByUsername, que el framework invoca para obtener un UserDetails a partir de un identificador. En Factullama, la entidad User implementa UserDetails directamente —decisión pragmática que evita una clase adaptadora extra, aunque acopla el modelo de dominio a Spring Security— y el filtro JWT consulta UserRepository.findByEmail() en lugar de pasar por un UserDetailsService explícito.

P6. ¿Qué hacen el AuthenticationManager y el AuthenticationProvider?

R: AuthenticationManager es la interfaz central que recibe un objeto Authentication sin verificar (en este caso, UsernamePasswordAuthenticationToken con email y password) y devuelve uno verificado, o lanza una excepción. Delegar la verificación real en uno o más AuthenticationProvider: DaoAuthenticationProvider, el que usa este proyecto, carga el UserDetails vía UserDetailsService y compara el password recibido contra el hash guardado usando el PasswordEncoder configurado (BCrypt). AuthService.authenticate() llama exactamente a authenticationManager.authenticate(...) antes de generar el JWT.

P7. ¿Por qué se deshabilita CSRF en una API JWT, y cuándo sí hace falta?

R: CSRF (Cross-Site Request Forgery) explota el hecho de que el navegador envía automáticamente las cookies de sesión en cualquier request, incluso si lo origina un sitio malicioso. Si la autenticación no usa cookies —como en Factullama, donde el cliente debe adjuntar manualmente el header Authorization: Bearer— no hay nada que el navegador envíe automáticamente, así que el ataque no aplica y CSRF puede deshabilitarse con seguridad. CSRF sí es necesario en aplicaciones que se autentican con cookies de sesión (apps server-rendered tradicionales).

P8. ¿Qué diferencia hay entre CORS y CSRF?

R: Son cosas distintas que suelen confundirse. CORS (Cross-Origin Resource Sharing) es un mecanismo del navegador para permitir o bloquear que JavaScript de un origen (dominio/puerto) llame a una API en otro origen —lo configura el servidor declarando qué orígenes confía. CSRF es un vector de ataque, no un mecanismo de protección. Factullama configura CORS explícitamente en SecurityConfig (localhost:4200, factullama.site) para que el frontend Angular, servido desde otro origen, pueda consumir la API; y deshabilita CSRF porque, como se explicó arriba, no aplica a su esquema de autenticación basado en tokens.

P9. ¿Cómo se almacenan las contraseñas de forma segura? ¿Qué hace BCrypt?

R: Nunca en texto plano ni con un hash rápido como MD5/SHA-1/SHA-256 sin sal, porque son vulnerables a ataques de fuerza bruta y tablas precomputadas (rainbow tables). BCrypt incorpora una sal aleatoria automáticamente y, sobre todo, es deliberadamente lento (tiene un "work factor" configurable) para que probar millones de contraseñas por segundo sea computacionalmente inviable, incluso si la base de datos se filtra. AuthService.register() guarda passwordEncoder.encode(request.getPassword()) usando BCryptPasswordEncoder, y la verificación ocurre dentro de DaoAuthenticationProvider, nunca comparando strings manualmente.

P10. ¿Cómo asegurarías un endpoint para que solo un rol ADMIN pueda acceder?

R: Hay dos formas principales y se pueden combinar: a nivel de configuración global, en el SecurityFilterChain con .requestMatchers("/api/admin/**").hasRole("ADMIN"); o a nivel de método, con @PreAuthorize("hasRole('ADMIN')") sobre el método del controller o servicio (requiere @EnableMethodSecurity). En Factullama, Role es un enum simple (ADMIN, USER) y User.getAuthorities() devuelve "ROLE_" + role.name(), que es exactamente el formato que Spring Security espera para que hasRole("ADMIN") funcione sin configuración adicional.

P11. ¿Qué riesgo real hay en guardar el secreto de firma JWT en un archivo de propiedades sin cifrar, y cómo lo solucionarías?

R: Si alguien obtiene ese secreto —por ejemplo porque quedó committeado en el repositorio, como pasa hoy en application-prod.properties de Factullama— puede firmar tokens JWT válidos para cualquier usuario, incluyendo un administrador, sin necesidad de conocer ninguna contraseña: control total sobre la autenticación del sistema. La solución es inyectar el secreto desde una variable de entorno o un gestor de secretos (no en el archivo versionado), rotarlo periódicamente, y considerar claves asimétricas (RS256) si varios servicios necesitan verificar tokens sin poder firmarlos.

P12. ¿Qué es OAuth2/OIDC y en qué se diferencia de la autenticación JWT propia de este proyecto?

R: OAuth2 es un protocolo de autorización delegada (permite que una aplicación acceda a recursos en nombre de un usuario sin conocer su password, típicamente vía un proveedor externo como Google) y OpenID Connect (OIDC) es la capa de autenticación construida encima que añade el concepto de identidad verificada (ID Token). Factullama no usa OAuth2: implementa su propio flujo de login con email/password y emite sus propios JWT, lo cual es válido para una app con su propio sistema de usuarios, pero no delega identidad a un tercero ni sigue el flujo de authorization code que usaría un "Iniciar sesión con Google". Es útil aclarar esta diferencia para no dar la impresión de que el proyecto usa OAuth2 cuando en realidad implementa JWT artesanal.

P13. Los JWT son stateless por diseño: ¿cómo manejarías un logout real o la revocación de un token?

R: Estrictamente, un JWT no se puede "invalidar" del lado del servidor sin guardar algún estado, porque su validez depende solo de la firma y la fecha de expiración. Las estrategias prácticas son: (1) mantener una blacklist de tokens revocados (en Redis, con TTL igual al tiempo de expiración restante del token) y chequearla en el filtro; (2) usar tokens de vida muy corta (minutos) combinados con refresh tokens revocables, de forma que el daño de un token robado esté acotado; o (3), del lado del cliente, simplemente borrar el token almacenado, aceptando que sigue siendo técnicamente válido hasta su expiración. Factullama hoy no implementa ninguna de estas — el logout es responsabilidad del cliente, que descarta el token.

4. LRBA (BBVA) — Preguntas y respuestas

Aclaración importante antes de esta sección

LRBA es una arquitectura/herramienta interna de BBVA para procesamiento batch (procesos masivos no interactivos: cierre de operaciones, reportes regulatorios, integración con sistemas legados del banco), construida típicamente sobre Java y procesamiento distribuido de datos. Al ser interna, no existe documentación pública oficial — lo que sigue es la mejor reconstrucción posible a partir de fuentes públicas (ofertas de empleo y descripciones de proyectos que la mencionan) más los conceptos estándar de la industria que casi con certeza subyacen a la herramienta. No la presentes como si la hubieras usado: presentala como lo que realmente es, conocimiento de los **fundamentos** de batch processing aplicables a cualquier herramienta concreta que el banco use internamente.

P1. ¿Qué es el procesamiento batch y en qué se diferencia del procesamiento online/síncrono?

R: El procesamiento online responde a una petición puntual de un usuario o sistema en tiempo real (milisegundos a segundos), como un login o crear una factura. El procesamiento batch ejecuta una tarea sobre un volumen grande de datos de forma planificada y no interactiva —por ejemplo, cerrar todas las operaciones del día, generar un reporte regulatorio, o conciliar movimientos contra un sistema legado— normalmente fuera de horario pico, optimizado para throughput total en vez de latencia individual. En banca, los procesos batch suelen ser críticos porque alimentan reportes obligatorios ante el regulador con plazos estrictos.

P2. ¿Qué es Spring Batch y cuáles son sus componentes principales?

R: Es el framework estándar de Spring para construir procesos batch robustos. Un Job es la unidad de trabajo completa, compuesta por uno o más Step. Cada Step normalmente sigue el patrón ItemReader (lee datos de origen: BD, archivo, otra API) → ItemProcessor (transforma/valida/filtra cada item) → ItemWriter (persiste el resultado), procesados en chunks (lotes de N items) para no cargar todo el dataset en memoria. Spring Batch también gestiona metadata de ejecución (JobRepository) para saber qué se procesó, lo cual habilita reintentos y reanudación.

P3. ¿Cómo se garantiza la idempotencia en un proceso batch que puede reintentarse?

R: Idempotencia significa que ejecutar el mismo proceso dos veces sobre los mismos datos produce el mismo resultado final, sin duplicar efectos (por ejemplo, no generar dos asientos contables por el mismo movimiento). Se logra típicamente marcando cada registro procesado con un estado o timestamp que el siguiente intento puede chequear antes de reprocesar, usando claves de negocio únicas con restricciones UNIQUE en la base de destino para que una inserción duplicada falle de forma controlada, o registrando explícitamente qué chunk/página ya se confirmó para reanudar exactamente donde quedó.

P4. ¿Cómo manejarías grandes volúmenes de datos sin saturar la memoria?

R: El principio es nunca cargar el dataset completo en memoria: se procesa en chunks/páginas de tamaño fijo (por ejemplo, leer y procesar de 500 en 500 registros, hacer commit, liberar memoria, seguir con el siguiente lote). En JPA esto se traduce en evitar findAll() sin paginación y usar Pageable o streaming de resultados (Stream con @QueryHints para fetch size), y en limpiar el contexto de persistencia (entityManager.clear()) periódicamente para que Hibernate no acumule miles de entidades gestionadas en memoria durante una transacción larga.

P5. ¿Qué estrategias de paralelización conocés para acelerar un proceso batch?

R: Partitioning: dividir el dataset total en particiones independientes (por rango de fechas, por región, por hash de una clave) y procesarlas en paralelo, cada una con su propio reader/processor/writer. Multi-threaded step: paralelizar el procesamiento dentro de un mismo step con varios hilos sobre la misma fuente de datos. Remote chunking/partitioning: distribuir el trabajo entre varios nodos/JVMs cuando un solo servidor no alcanza, comunicándose vía una cola de mensajes. La elección depende de si el cuello de botella es CPU, I/O, o si los datos se pueden particionar de forma natural sin dependencias entre particiones.

P6. ¿Cómo gestionarías errores parciales en un batch de millones de registros?

R: Definiendo de antemano una política explícita: skip (saltar el registro con error y continuar, registrándolo para revisión manual posterior, hasta un límite máximo de skips), retry (reintentar automáticamente errores transitorios como un timeout de red, con backoff), y un mecanismo de dead-letter (mover los registros que fallan definitivamente a una tabla o cola separada para investigación, sin bloquear el procesamiento del resto). Lo crítico es que un solo registro corrupto no debería abortar un proceso que afecta a millones de registros válidos, pero tampoco debería "desaparecer" silenciosamente sin dejar rastro auditable.

P7. ¿Qué significa que un proceso batch sea reanudable (restartable) y por qué importa?

R: Que si el proceso se interrumpe a mitad de camino (caída del servidor, corte de red, error no previsto), pueda reiniciarse y continuar desde el último punto consistente en vez de desde el principio o, peor, sin saber desde dónde. Esto requiere persistir metadata de progreso (qué chunk o página fue la última confirmada) en un store durable, no en memoria. En un contexto bancario donde un job puede tardar horas y procesar datos regulatorios, reprocesar todo desde cero por una caída puntual puede significar incumplir un plazo legal de reporte.

P8. ¿Qué consideraciones de integridad hay al integrar un batch con sistemas legados o bases de datos operacionales bancarias?

R: Evitar que el proceso batch compita por recursos con el sistema operacional en horario de alta demanda (por eso suelen correr de madrugada), no bloquear tablas críticas con locks largos, validar agresivamente los datos de entrada porque los sistemas legados suelen tener formatos heredados o inconsistencias históricas, y mantener trazabilidad completa (qué se leyó, qué se escribió, cuándo) porque en banca cualquier movimiento de datos suele estar sujeto a auditoría. También es común que el legado no tenga una API moderna, así que la integración se hace vía extracción a archivos planos, vistas de solo lectura, o colas de mensajería intermedias en vez de acceso directo.

P9. ¿Cómo planificarías/orquestarías la ejecución de jobs batch en un entorno empresarial?

R: Con un scheduler/orquestador dedicado en vez de un simple cron embebido en la aplicación, especialmente cuando hay dependencias entre jobs (el job B solo debe correr si el job A terminó con éxito). En banca es común usar herramientas como Control-M o un orquestador propio, que además dan visibilidad centralizada, alertas si un job se retrasa o falla, y reintentos automáticos según política. A nivel de Spring, JobLauncher y JobRepository proveen la base, pero la orquestación entre múltiples jobs y sistemas suele vivir un nivel por encima de la aplicación individual.

5. APX / Arquitectura Plataforma eXtendida (BBVA) — Preguntas y respuestas

Corrección y aclaración importante

El nombre correcto del estándar es **APX** (Arquitectura Plataforma eXtendida), no "AXP" — vale la pena usar el nombre correcto en la entrevista, suma puntos de credibilidad. Es la plataforma interna de BBVA para construir microservicios backend en Java/Spring Boot sobre arquitectura hexagonal (puertos y adaptadores), con soporte tanto para ejecución síncrona (online, vía REST) como asíncrona (batch/eventos), pensada para desplegarse en una plataforma cloud/PaaS interna con pipelines de CI/CD. Igual que con LRBA, no es información pública oficial: lo siguiente son los fundamentos de arquitectura de software que casi con certeza sustentan APX, presentados con honestidad como conocimiento conceptual, no como experiencia directa con la herramienta.

P1. ¿Qué es la arquitectura hexagonal y qué problema resuelve frente a una arquitectura en capas tradicional?

R: La arquitectura hexagonal (puertos y adaptadores, de Alistair Cockburn) pone el núcleo de negocio (el dominio) en el centro, completamente aislado de detalles técnicos como el framework web, la base de datos o sistemas externos. La comunicación entre el dominio y el exterior se hace siempre a través de interfaces (puertos), nunca por dependencia directa. El problema que resuelve es que, en una arquitectura en capas tradicional, la lógica de negocio termina dependiendo directamente de JPA, de Spring MVC o de un SDK externo, lo que dificulta testear el dominio en aislamiento y encarece cambiar de tecnología (por ejemplo, migrar de REST a mensajería, o de una base de datos a otra) sin tocar las reglas de negocio.

P2. ¿Qué son los puertos y los adaptadores?

R: Un puerto es una interfaz definida por el dominio que describe una capacidad que necesita (un puerto de salida, ej. "GuardarFactura") o que ofrece (un puerto de entrada, ej. "CrearFactura"), sin saber nada de cómo se implementa. Un adaptador es la implementación concreta de ese puerto con una tecnología específica: un adaptador de salida podría ser una clase que implementa GuardarFactura usando JPA/Hibernate contra MariaDB, y un adaptador de entrada podría ser un controller REST que traduce una petición HTTP en una llamada al puerto de entrada del dominio. Esto permite cambiar de MariaDB a otra base, o de REST a un consumidor de eventos, sin tocar una sola línea de las reglas de negocio.

P3. ¿Cómo se traduce la arquitectura hexagonal en capas Domain / Application / Infrastructure?

R: Domain contiene las entidades y reglas de negocio puras, sin ninguna dependencia de frameworks (ni anotaciones de Spring, ni de JPA). Application orquesta los casos de uso (qué pasos hay que seguir para "emitir una factura"), define los puertos que necesita, y depende solo del dominio. Infrastructure contiene todos los adaptadores concretos: controllers REST, repositorios JPA, clientes HTTP a servicios externos, configuración de Spring. La regla de dependencia es siempre hacia el centro: Infrastructure puede depender de Application y Domain, pero Domain no puede depender de nada de afuera. Es la inversión del flujo de dependencias respecto a una arquitectura en capas clásica, donde la lógica de negocio suele terminar acoplada a JPA por comodidad.

P4. ¿Qué diferencia hay entre ejecución síncrona y asíncrona en una API, y cuándo usarías cada una?

R: Síncrona: el cliente espera la respuesta en la misma conexión/request, apropiado cuando el resultado se necesita inmediatamente y el procesamiento es rápido (consultar el estado de una factura). Asíncrona: el sistema acepta la petición, devuelve una confirmación inmediata (o un identificador de seguimiento) y procesa el trabajo real en background, normalmente vía una cola de mensajes o un job batch, notificando el resultado más tarde (webhook, polling, evento). Se usa asíncrono cuando el procesamiento es largo, cuando hay que desacoplar la disponibilidad del cliente de la del sistema downstream, o cuando se necesita resiliencia ante picos de carga sin saturar threads esperando.

P5. ¿Qué es una arquitectura de microservicios y qué retos introduce?

R: Es dividir un sistema en servicios pequeños, desplegables de forma independiente, cada uno responsable de un dominio de negocio acotado, comunicándose por red (REST, mensajería) en vez de en memoria. Los retos principales son: consistencia de datos distribuida (no hay transacciones ACID entre servicios, hay que pensar en consistencia eventual o patrones como Saga), comunicación (latencia de red, manejo de fallos parciales con timeouts/circuit breakers), observabilidad (trazar una petición que atraviesa varios servicios requiere logging y tracing distribuido), y la complejidad operacional de desplegar, versionar y monitorear muchos servicios en vez de uno solo.

P6. ¿Cómo diseñarías una API RESTful bien hecha?

R: Modelando recursos como sustantivos en la URL (/facturas, /facturas/{id}) y usando los verbos HTTP según su semántica (GET para leer sin efectos secundarios, POST para crear, PUT/PATCH para actualizar, DELETE para eliminar), devolviendo códigos de estado correctos (201 al crear, 204 sin contenido, 404 si no existe, 400 si la petición es inválida, 409 si hay conflicto), versionando la API cuando hay cambios incompatibles (/api/v1/...), y siendo consistente en el formato de error en toda la API — exactamente el patrón de ApiError que usa el GlobalExceptionHandler de Factullama.

P7. ¿Qué es CI/CD y cómo lo aplicarías a microservicios en una plataforma PaaS?

R: CI (integración continua) es automatizar build, análisis estático y tests en cada cambio de código para detectar problemas temprano. CD (entrega/despliegue continuo) automatiza llevar ese build validado hasta un entorno (staging o producción) sin intervención manual, normalmente con promoción progresiva entre entornos y la posibilidad de rollback rápido. En una plataforma PaaS interna, el pipeline típico es: compilar y testear, construir una imagen de contenedor, escanear vulnerabilidades, desplegar a un entorno de pruebas automatizadas, y promover a producción con aprobación o automáticamente si pasan los checks de calidad.

P8. ¿Cómo testearías un microservicio construido con arquitectura hexagonal?

R: El dominio se testea con tests unitarios puros, sin mocks de infraestructura, porque no tiene dependencias externas — son los tests más rápidos y más valiosos. Los casos de uso de Application se testean mockeando los puertos (interfaces), verificando que se orquestan correctamente sin necesidad de una base de datos real. Los adaptadores de Infraestructura (el repositorio JPA real, el cliente HTTP real) se testean con tests de integración, idealmente con Testcontainers para levantar una base de datos real en un contenedor en vez de usar mocks o H2, que pueden esconder diferencias de comportamiento con la base de producción.

P9. ¿Qué ventaja concreta tiene desacoplar el dominio de negocio de frameworks como Spring?

R: El dominio se puede testear en milisegundos sin levantar ningún contexto de Spring ni base de datos, lo que acelera el feedback loop de desarrollo. También facilita migrar de framework o de tecnología de persistencia sin reescribir las reglas de negocio, porque esas reglas no conocen ni Spring ni JPA. En Factullama, en cambio, la entidad User implementa UserDetails de Spring Security directamente — una decisión pragmática válida para un proyecto de este tamaño, pero que es justamente el tipo de acoplamiento que arquitecturas como APX buscan evitar en sistemas más grandes y de mayor vida útil, separando el modelo de dominio puro de los adaptadores de seguridad.

6. Cómo hablar de LRBA y APX sin experiencia directa

Ya fuiste transparente con la empresa: dijiste que no manejas LRBA ni APX pero que te adaptas rápido. Esa honestidad es un activo, no algo que disimular — el objetivo de esta sección es darte un guion para reforzarla con evidencia concreta de que aprendes rápido, en vez de quedarte en una afirmación vacía.

6.1 Guion sugerido si preguntan directamente por LRBA o APX

«Te comenté desde el inicio que no tengo experiencia directa con LRBA/APX porque son herramientas internas de BBVA — no podía tenerla sin haber trabajado antes en el banco, y prefiero ser claro en eso a fingir. Lo que sí tengo son las bases sobre las que se construyen: Java y Spring Boot a nivel profesional, una arquitectura en capas con separación clara entre controller, servicio y repositorio en Factullama, manejo de transacciones, y trabajo con procesos que requieren precisión —como la firma criptográfica de documentos para SUNAT—, que es el mismo tipo de cuidado que exige un proceso batch regulatorio. Mi expectativa realista es que los primeros días los voy a invertir en entender la herramienta específica y sus convenciones internas, pero los conceptos de fondo —procesamiento por lotes, arquitectura hexagonal, microservicios síncronos/asíncronos— ya los manejo, así que la curva de aprendizaje debería ser de adaptación a una herramienta concreta, no de aprender un paradigma desde cero.»

6.2 Tres formas de demostrar adaptabilidad con evidencia, no solo afirmarla

a) Menciona una vez que ya migraste de paradigma rápido.

R: En Factullama tuviste que aprender de cero firma digital XML (Apache Santuario, XMLSignature, certificados PKCS12) y la generación de UBL 2.1 sin haber trabajado antes con ninguna de las dos. Es un ejemplo real y verificable de absorber un dominio técnico desconocido y específico de una industria regulada (facturación electrónica peruana) en el contexto de un proyecto propio.

b) Tendé puentes explícitos entre lo que sabés y lo que se espera de LRBA/APX.

R: No digas solo «puedo aprender» — decí en qué se parece lo que ya hiciste: «la separación Controller/Service/Repository que usé es la misma idea de fondo que Domain/Application/Infraestructure en hexagonal, solo que menos estricta en las dependencias» o «calcular IGV con BigDecimal y manejar transacciones con @Transactional es el mismo nivel de cuidado que un proceso batch financiero necesita, aplicado a una escala distinta».

c) Hacé una pregunta inteligente de vuelta sobre la herramienta.

R: Mostrar curiosidad genuina («¿LRBA se apoya en Spring Batch o es una solución propia sobre Spark /procesamiento distribuido?», «¿APX define los puertos como interfaces Java estándar o usa algún generador/contrato propio?») demuestra que ya pensaste en la arquitectura conceptual antes de la entrevista, en vez de llegar sin haber investigado nada al respecto.

6.3 Qué NO hacer

Evitá improvisar detalles específicos de LRBA o APX como si los hubieras usado — si te preguntan algo muy puntual de la implementación interna (un nombre de clase propia, un detalle de configuración exclusivo del banco) y no lo sabés, decilo directamente: «eso específico no lo conozco, es interno de la herramienta, pero el concepto general detrás es...» y volvés a terreno firme. Es mucho más sólido que una respuesta vaga que suene a relleno.

